



2205014



2205014_build



2205014_exam_practice_problems



2205014_scope_table



2205014_symbol_info



2205014_build

Inside the .md file there are 7 problems. Problem 7 is incomplete so you don't have to complete it. Just add at least 4 more problems whose category is hard.

Problem 8 — Rename and Rehash (R) · Hard

New command: `R <old_key> <new_key>` — exactly two arguments.

Task: Locate `<old_key>` *only* within the current (innermost) ScopeTable. If found, change its name to `<new_key>`. Because the name changes, its SDBM hash changes as well. You must unlink the symbol node from its current bucket chain and re-insert it into the appropriate bucket chain of the **same** ScopeTable using the new hash value.

Edge Cases & Rules:

- If `<old_key>` does not exist in the current ScopeTable, print a "not found" error and do nothing.
- If `<new_key>` *already exists* anywhere within the current ScopeTable, the rename operation must be aborted entirely to prevent duplicate keys in the same scope, and a conflict message must be printed.
- Do not allocate a new `SymbolInfo` node; you must manipulate the pointers of the existing node to safely re-link it.

Exact output:

- On successful rename:

' <old_key>' renamed to '<new_key>' and rehashed into bucket <new_bucket>

- If `<old_key>` is missing:

```
'<old_key>' not found in the current ScopeTable
```

- o If `<new_key>` is already taken:

```
Rename failed: '<new_key>' already exists in the current ScopeTable
```

Argument mismatch: If parameters are fewer than 2 or more than 2, print `Number of parameters mismatch for the command R`.

Hint: First check if `<new_key>` exists in the current scope using `lookUp`. If it does, abort. Then, write a method inside `ScopeTable` that manually walks the bucket chain of `<old_key>`, unlinks it from the linked list (handling whether it's the head or middle/end of the chain), updates its internal name field via `setName()`, computes the new SDBM hash with `<new_key>`, and appends it to the end of the new bucket's chain.

Sample Input

```
4
I x VAR
I y VAR
R x z
R w k
R y z
P C
Q
```

Sample Output

```
ScopeTable# 1 created
Cmd 1: I x VAR
      Inserted in ScopeTable# 1 at position 1, 1
Cmd 2: I y VAR
      Inserted in ScopeTable# 1 at position 2, 1
```

```

Cmd 3: R x z
      'x' renamed to 'z' and rehashed into bucket 3
Cmd 4: R w k
      'w' not found in the current ScopeTable
Cmd 5: R y z
      Rename failed: 'z' already exists in the current ScopeTable
Cmd 6: P C
      ScopeTable# 1
      1-->
      2--> <y,VAR>
      3--> <z,VAR>
      4-->
Cmd 7: Q
      ScopeTable# 1 removed

```

Problem 9 — Flatten Scope Chain (`z`) · *Hard*

New command: `z` — takes no arguments.

Task: Collapse the entire active chain of scopes down into the root scope (`ScopeTable# 1`). This is a global consolidation operation. To respect variable shadowing, symbols must be migrated starting from the **innermost** (current) scope moving outward to the root.

Rules:

- Walk each scope table from `current` down to the root. For each table, iterate through its buckets and chains.
- If a symbol's name does not yet exist in the root scope (`ScopeTable# 1`), insert it into the root scope.
- If a symbol's name *already* exists in the root scope, overwrite the existing symbol's definition with the inner scope's definition to correctly preserve shadowing behavior in the final flattened table.
- After all symbols from all intermediate scopes have been processed and filtered into `ScopeTable# 1` , delete all intermediate `ScopeTable` objects to prevent memory leaks, and reset the `SymbolTable`'s `current` pointer back to the root scope.

Exact output:

- If executed at the root scope (nothing to flatten):

Already at root scope. Cannot flatten chain.

- o If executed from an inner scope:

Flattening scope chain into ScopeTable# 1...
Moved '<key>' from ScopeTable# <U> to ScopeTable# 1 at position <bud
Scope chain flattened. Current scope reset to ScopeTable# 1.

Argument mismatch: If a token follows **Z**, print **Number of parameters mismatch for the command Z**.

Sample Input

```
4
I a GLOBAL
S
I b LOCAL_SUB
I a LOCAL_SHADOW
S
I c DEEP_LOCAL
Z
P C
Q
```

Sample Output

```
ScopeTable# 1 created
Cmd 1: I a GLOBAL
      Inserted in ScopeTable# 1 at position 2, 1
Cmd 2: S
      ScopeTable# 2 created
Cmd 3: I b LOCAL_SUB
      Inserted in ScopeTable# 2 at position 3, 1
Cmd 4: I a LOCAL_SHADOW
```

```

Inserted in ScopeTable# 2 at position 2, 1
Cmd 5: S
    ScopeTable# 3 created
Cmd 6: I c DEEP_LOCAL
    Inserted in ScopeTable# 3 at position 4, 1
Cmd 7: Z
    Flattening scope chain into ScopeTable# 1...
    Moved 'c' from ScopeTable# 3 to ScopeTable# 1 at position 4, 1
    Moved 'b' from ScopeTable# 2 to ScopeTable# 1 at position 3, 1
    Moved 'a' from ScopeTable# 2 to ScopeTable# 1 at position 2, 1
    ScopeTable# 3 removed
    ScopeTable# 2 removed
    Scope chain flattened. Current scope reset to ScopeTable# 1.
Cmd 8: P C
    ScopeTable# 1
    1-->
    2--> <a,LOCAL_SHADOW>
    3--> <b,LOCAL_SUB>
    4--> <c,DEEP_LOCAL>
Cmd 9: Q
    ScopeTable# 1 removed

```

Problem 10 — Scope Selective Split (x) · Hard

New command: `X <type>` — exactly one argument.

Task: Filter and split the current scope table based on a symbol type. A new scope table is created as a child of the current scope, and all symbols in the current scope whose type exactly matches `<type>` are extracted and moved into this new scope table. The new scope table then becomes the active `current` scope.

Rules:

- Scan the current scope table for entries matching `<type>`.
- If no symbols match the given `<type>`, do not create a new scope; print an error message instead.
- If matches exist, allocate a new `ScopeTable` with the standard bucket size, establish the old scope as its parent, and move all matching symbol nodes from the old scope into the new one (rehashing them into their new buckets).
- The order of nodes within the chains must be preserved during migration.

Exact output:

- If symbols are successfully split into a new scope:

```
ScopeTable# <newU> created via selective split
Moved '<key>' to ScopeTable# <newU> at position <bucket>, <pos>
```

- If no matching symbols are found:

```
Split failed: No symbols found with type '<type>'
```

Argument mismatch: If parameters are missing or extra tokens are given, print
 Number of parameters mismatch for the command X.

Sample Input

```
3
I a VAR
I b FUNC
I c VAR
X VAR
P A
Q
```

Sample Output

```
ScopeTable# 1 created
Cmd 1: I a VAR
      Inserted in ScopeTable# 1 at position 2, 1
Cmd 2: I b FUNC
      Inserted in ScopeTable# 1 at position 3, 1
Cmd 3: I c VAR
      Inserted in ScopeTable# 1 at position 1, 1
Cmd 4: X VAR
      ScopeTable# 2 created via selective split
      Moved 'c' to ScopeTable# 2 at position 1, 1
      Moved 'a' to ScopeTable# 2 at position 2, 1
Cmd 5: P A
      ScopeTable# 2
```

```

1--> <c,VAR>
2--> <a,VAR>
3-->
      ScopeTable# 1
      1-->
      2-->
      3--> <b,FUNC>

```

Cmd 6: Q

```

ScopeTable# 2 removed
ScopeTable# 1 removed

```

Problem 11 — Global Promotion (0) · Hard

New command: 0 <key> — exactly one argument.

Task: Look up <key> using the standard nearest-scope rules. If the symbol is found in an inner active scope, "promote" it by moving it out of its current scope table and inserting it directly into the root scope (ScopeTable# 1). This makes the symbol globally visible across all scopes.

Rules:

- Locate the symbol using a standard chain lookup. If not found anywhere, print an error.
- If the symbol is found and it is already located in ScopeTable# 1 , print a message indicating it is already global.
- If the symbol is found in an inner scope (ScopeTable# > 1), check if the root scope (ScopeTable# 1) already contains a symbol with that name. If a naming conflict exists in the root scope, abort the promotion and print a conflict error.
- If clear, unlink the symbol from its inner scope table and rehash/insert it into ScopeTable# 1 .

Exact output:

- On successful promotion:

```
Promoted '<key>' from ScopeTable# <U> to global ScopeTable# 1
```

- If already in root scope:

```
Promotion skipped: '<key>' is already in the root ScopeTable
```

- o If a conflict exists in the root scope:

```
Promotion failed: '<key>' already exists in the root ScopeTable
```

- o If not found anywhere:

```
'<key>' not found in any of the ScopeTables
```

Argument mismatch: If parameters are incorrect, print `Number of parameters mismatch for the command 0`.

Sample Input

```
3
I x GLOBAL_VAR
S
I y LOCAL_VAR
I x LOCAL_SHADOW
S
I z DEEP_VAR
O z
O y
O x
Q
```

Sample Output

```
ScopeTable# 1 created
Cmd 1: I x GLOBAL_VAR
```

```
    Inserted in ScopeTable# 1 at position 1, 1
Cmd 2: S
    ScopeTable# 2 created
Cmd 3: I y LOCAL_VAR
    Inserted in ScopeTable# 2 at position 2, 1
Cmd 4: I x LOCAL_SHADOW
    Inserted in ScopeTable# 2 at position 1, 1
Cmd 5: S
    ScopeTable# 3 created
Cmd 6: I z DEEP_VAR
    Inserted in ScopeTable# 3 at position 3, 1
Cmd 7: O z
    Promoted 'z' from ScopeTable# 3 to global ScopeTable# 1
Cmd 8: O y
    Promoted 'y' from ScopeTable# 2 to global ScopeTable# 1
Cmd 9: O x
    Promotion failed: 'x' already exists in the root ScopeTable
Cmd 10: Q
    ScopeTable# 3 removed
    ScopeTable# 2 removed
    ScopeTable# 1 removed
```